专题一习题选讲

程序设计实践

计算机与通信工程学院

模拟内存管理

设计并开发一个实验内存管理器，实现对内存的分配、释放和整理。对应的接口为new、del和def，使用语法为：

new size：**分配size字节大小的内存块**，返回该内存块的句柄handle（句柄即内存块编号，分配的第一个内存块编号为1，以此类推），size为正整数。

del handle：**释放**句柄handle指向的内存块。

def：整理内存碎片，**将所有已分配内存块按地址从低到高的顺序迁移**，使空闲内存碎片在高端地址端拼接在一起。

样例

【输入形式】

输入中有多组测试数据。每组测试数据的第一行为两个正整数T和MaxMem

（1<=T<=10000，1<=MaxMem<=10000），其中T为操作次数，MaxMem为初始内存大小，随后有T行操作指令。

【输出形式】

对每组测试数据，按操作顺序输出操作结果。对每个new操作，在单独行中输出结果，成功时输出其返回句柄值，失败则输出NULL。

若del操作失败，输出ILLEGAL_OPERATION。

def不产生输出。

【样例输入】

```
6 10
new 5
new 3
del 1
new 6
def
new 6
```

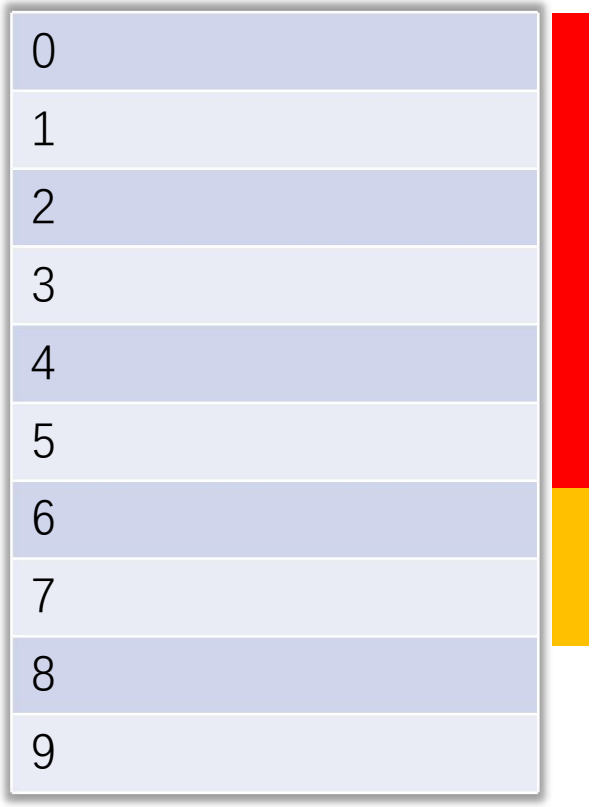
【样例输出】

```
1
2
NULL
3
```

题目分析:

模拟内存的三种行为。

- **new size** 在已分配区域的末尾追加新的内存块。
- **del** handle 释放句柄（编号）为handle的内存块。
- **def** 将内存块整理压缩，紧密地排列。



题目的歧义:

在两个内存块中间有足够的内存区域，那么新分配的内存可以用吗?

答：不使用中间的内存区域，只在末尾分配新的内存块。

步骤演示-new

指令:
new 6
new 2
new 4

输出
1
2
NULL

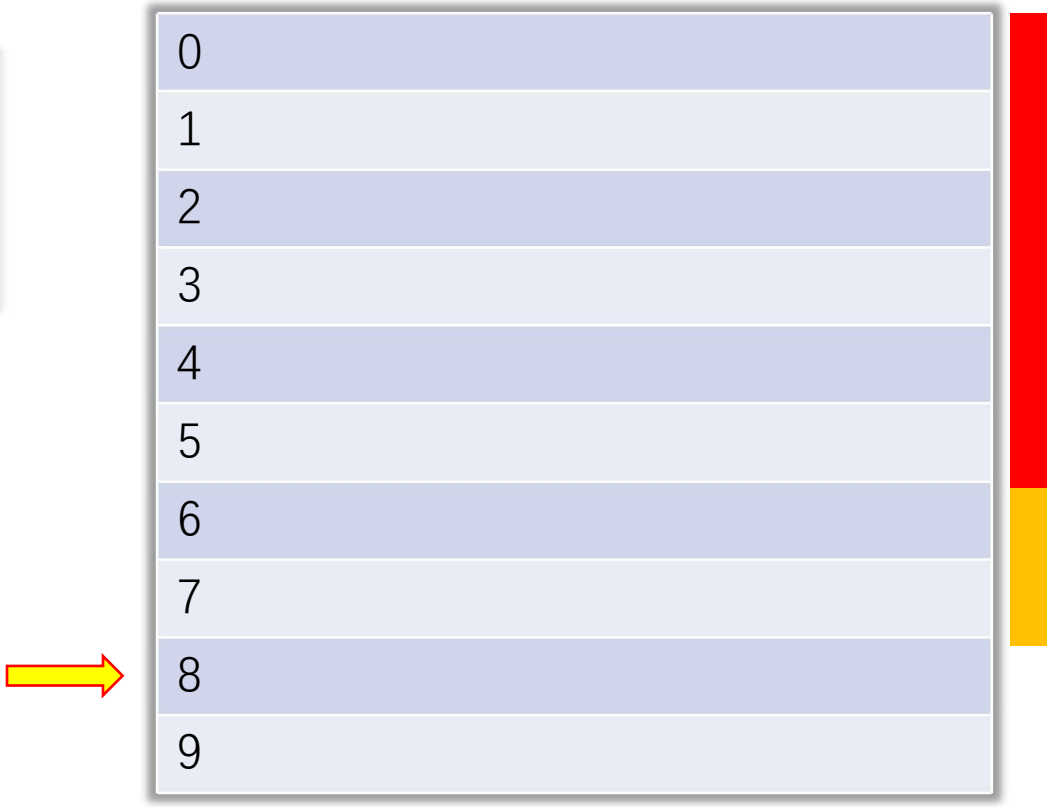
超出限制

0
1
2
3
4
5
6
7
8
9

步骤演示-del

指令:
del 1
del 1 //没有句柄为1的内存块了

输出
ILLEGAL_OPERATION



步骤演示-def

指令:
def

输出

0
1
2
3
4
5
6
7
8
9

步骤演示-def

指令:
def

输出

0
1
2
3
4
5
6
7
8
9

解题方法一

- 数组存储

```
//定义一个数组模拟内存，定义一个指针指向占用过的内存末尾
int t; //操作指令数
int maxMem;//记录最大内存
int kuai[10001];//记录index为handle的连续内存的大小
int handleStart[10000];//记录index 为handle的内存开始的位置

int ram[10005];//记录内存是否被占用，1表示被占用了
int tail = 0;//记录当前最后一块内存的下标
int total;//已经使用的内存块大小
bool del[10005];//记录index 为handle的内存能不能删除
```

- **New操作**

1. n 已分配内存数量
2. *if* 内存空间足够分配

   分配内存编号：n+1

   记录内存长度：kuai[n]=x;

   申请内存空间：ram[1:x]

   记录新内存开始位置：

   handleStart[n]=tail

   更新尾指针：tail+=x;

   else 内存空间不够
3. *output*

```
if ((maxMem - tail) >= x) ];//空闲内存足够分配
        {
            total +=x;

            kuai[++n] =x;
            //handle n的内存大小

            for (int i = tail; i <kuai[n]; i++)
            {
                ram[i] = 1;
            }
            handleStart[n] = tail;

            tail += x;
            del[n] = 1;
            cout << n << endl;
            }

        else  //空闲内存不够分配
        {
            cout << "NULL"<<endl;
        }
    }
```

解题方法二

- 结构体存储

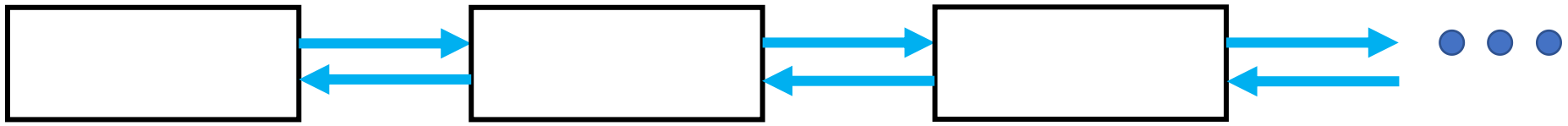
```
struct Mem
{
    int handle, start, len;
    Mem(int h, int s, int l) {
        handle = h;
        start = s;
        len = l;
    }
    Mem(){}

};//内存块设置


vector<Mem>Memory;      //存储内存块信息
int end;                //指向已分配内存区域的末尾+1
int numberOfMem;        //已分配内存块数目，用于编号句柄
```

解题方法三

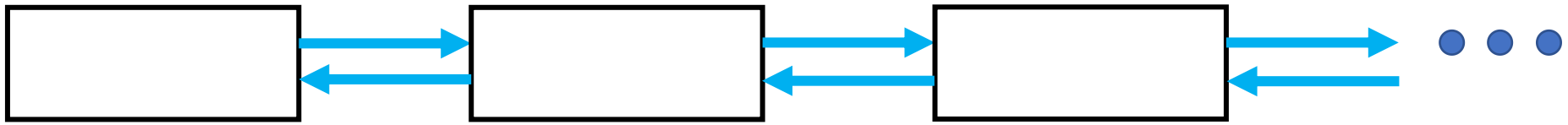
- 链表存储



```
struct MemoryFrag {
    int start;
    int len;
    MemoryFrag *prev, *next;
};
```

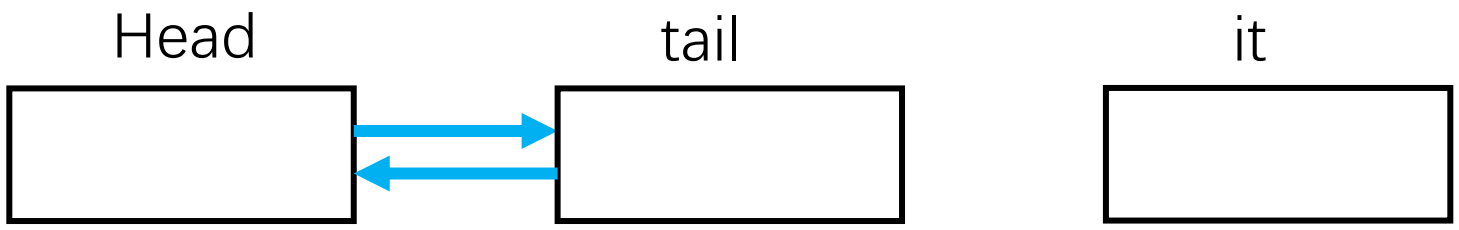
解题方法三

- 链表存储

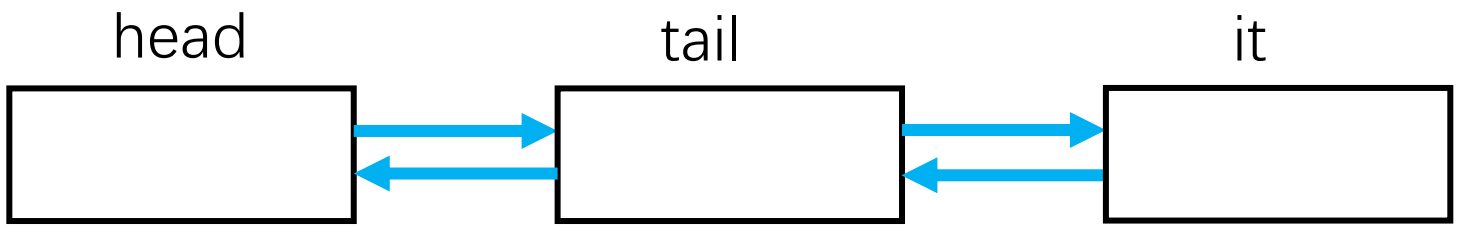


- **分配内存：** 在链表尾部加入新节点
- **释放内存：** 删除节点
- **内存整理：** this->start = this->prev->start + this->prev->len

在尾部添加节点



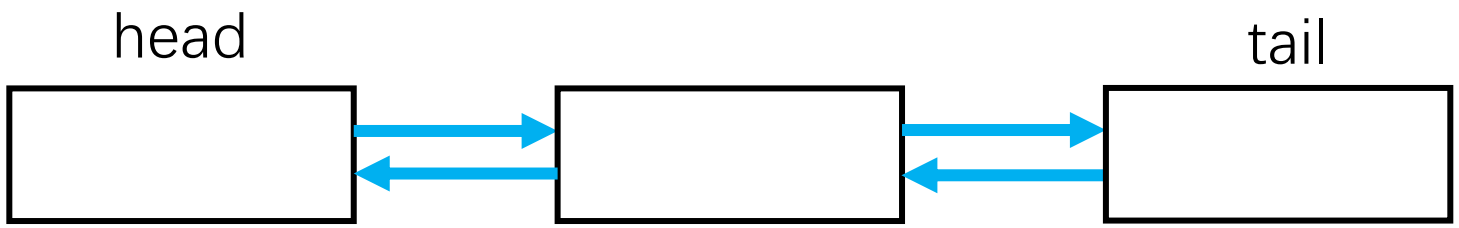
在尾部添加节点




```
it->prev = tail;

tail->next = it;
```

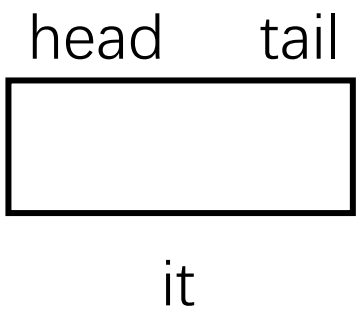
在尾部添加节点




```
it->prev = tail;
tail->next = it;

tail = it;
```

在尾部添加节点——如果链表为空?

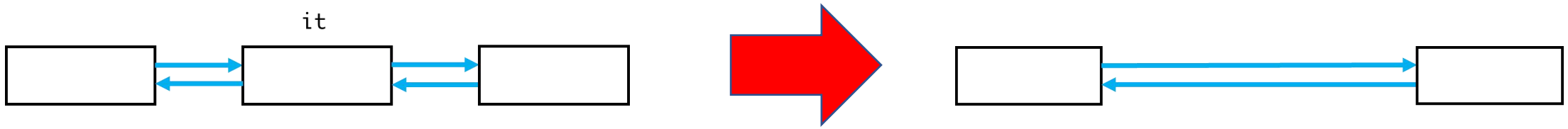


head = tail = it

如何删除节点？

删除节点

- 节点在中间




```
it->prev->next = it->next;

it->next->prev = it->prev;


delete it;
```

特殊情况

- it是head但不是tail

```
head = it->next;
it->next->prev = NULL;
delete it;
```

- it是tail但不是head

```
it->prev->next = NULL;
tail = it->prev;
delete it;
```

- it既是head又是tail

```
head = tail = NULL;
delete it;
```

Tell yourselves WHY they are right.

如何整理碎片？

如何整理碎片？

```
it->start = it->prev->start + it->prev->len;
```

这个节点存储的起始位置等于前一个节点的起始位置加前一个节点的长度。

留个**小问题**：对于head节点怎么处理呢？